XBRL GL Next Taxonomy Framework — Part 4: Aligned Pool and extension governance

Document status: Draft — For Collaborative Review

Date: 2026-09-15

Author: Nobuyuki SAMBUICHI

Role: Advisor to XBRL Japan

Project: XBRL GL Next

Distribution: Public collaborative review draft.

Repository version: public collaborative review draft.

Editors: SAMBUICHI, Nobuyuki

Contributors: invited project participants (individual names not yet confirmed for publication).

This document is a draft under development by Nobuyuki SAMBUICHI, Advisor to XBRL Japan, for collaborative review within the XBRL GL Next project. It is made available for public collaborative review. Copyright and license terms under review.

Status

This document is a collaborative review draft under development by Nobuyuki SAMBUICHI. It has not been approved or endorsed as an official specification by XBRL Japan, XBRL Europe, XBRL International, or any other organisation. Copyright and licence terms remain under review.

Abstract

This Part specifies the serialisation-independent governance and machine-readable artefacts used to classify, register, review, version, publish, and reuse Shared, Aligned, and Distinct extensions. It connects accepted FSM sources to Specialisation, Graph Walk, Post-Graph Walk, and the HMD-directed binding rules in Parts 2 and 3. A worked assembly extension adds the gl-btx:businessTransactions HMD alongside the gl-cor:accountingEntries HMD.

Table of Contents

Status 1

Abstract 1

1 Language independence 3

2 Introduction 3
- **2.1 Scope** 3
- **2.2 Normative language** 3
- **2.3 Relationship to the other Parts** 3
- **2.4 Implementation independence** 3

3 Terms and definitions 4

4 Extension classification 4
- **4.1 Shared** 4
- **4.2 Aligned** 4
- **4.3 Distinct** 4
- **4.4 Classification decision** 5

5 Governed artefacts 5
- **5.1 Aligned extension registry** 5
- **5.2 Aligned semantic library** 5
- **5.3 Publication information** 5

6 Submission and review procedure 5
- **6.1 Submission package** 5
- **6.2 Semantic review** 6
- **6.3 Processing evidence** 6
- **6.4 Binding evidence** 7

7 Lifecycle and compatibility 8
- **7.1 Versioning** 8
- **7.2 Deprecation and withdrawal** 8
- **7.3 Namespace and release date** 8

8 Worked assembly extension: businessTransactions (informative) 8
- **8.1 Baseline** 8
- **8.2 Extension objective** 8
- **8.3 Formal HMD root** 8
- **8.4 Required generated evidence** 9
- **8.5 Shared QName condition** 9
- **8.6 Acceptance information** 9
- **8.7 Taxonomy assembly evidence** 9

9 Conformance 9
- **9.1 Maintainer conformance** 9
- **9.2 Extension-user conformance** 10
- **9.3 Prohibited practices** 10

Appendix A (informative) Illustrative registry record for the assembly extension 11

Appendix B References 12

Appendix C Intellectual property status 13

Appendix D Document history 14

Appendix E (informative) Correspondence with XBRL GL Next Requirements 16

1 Language independence

This collaborative review draft is written in English. The preferred spelling convention is UK English. XML names, QNames, namespace URIs, and normative taxonomy filenames are case-sensitive and are not translated.

2 Introduction

2.1 Scope

This Part defines extension classification, registry data, semantic-library publication, review, lifecycle, and conformance. It governs the semantic inputs and published taxonomy results used by Parts 2 and 3. Processing interfaces, diagnostics, logs, execution records, checksums, and other ancillary output are implementation-specific.

2.2 Normative language

The key words "MUST", "MUST NOT", "SHOULD", "SHOULD NOT", and "MAY", when they appear in uppercase, are interpreted as described in RFC 2119. "MUST" and "MUST NOT" express requirements indispensable to artefact conformance or interoperability. "SHOULD" and "SHOULD NOT" express recommended courses for which a conforming implementation can use another documented course when justified. "MAY" expresses a permitted option. Other requirement language is descriptive.

2.3 Relationship to the other Parts

The XBRL GL Next Requirements Specification provides the background, design objectives, and requirements for the Framework.

The XBRL GL Next Taxonomy Framework consists of the following Parts:

- XBRL GL Next Taxonomy Framework — Part 1: General rules;
- XBRL GL Next Taxonomy Framework — Part 2: Tuple palette taxonomy;
- XBRL GL Next Taxonomy Framework — Part 3: OIM palette taxonomy;
- XBRL GL Next Taxonomy Framework — Part 4: Aligned Pool and extension governance;
- XBRL GL Next Taxonomy Framework — Part 5: Tavi palette taxonomy.

Part 1 defines the serialisation-independent semantic pipeline and the formal HMD contract. Part 2 defines the XBRL 2.1 Tuple realisation generated from formal HMDs. Part 3 defines the OIM realisation generated from the same HMDs. This Part 4 governs extension classification, registration, alignment and publication. Part 5 defines purpose-specific HMD views and their Tavi taxonomy and Table Template realisation.

This Part 4 specifies the governance of semantic extensions, including their classification as Shared, Aligned, or Distinct, their review and registration, and the management of the Aligned Pool and publication requirements.

Parts 2, 3 and 5 provide alternative taxonomy realisations of the common semantic framework established by Part 1, under the extension governance defined in Part 4. Each HMD serves as a purpose-specific palette, selecting reusable semantics from COR, BTX and AAA, which constitute the semantic paint boxes.

2.4 Implementation independence

The requirements in this Part define observable governed inputs, review outcomes, generated taxonomy evidence, and deterministic relationships among them.

The maintained semantic processing programs are `tools/semantic/specialisation.py`, `tools/semantic/graphwalk.py`, and `tools/semantic/post_graphwalk.py`; the sole taxonomy-generation authority is `tools/taxonomy/xBRLGL_TaxonomyGenerator.py`.

3 Terms and definitions

For the purposes of this document, the terms and definitions given in Part 1, Part 2, Part 3 and the following apply. Unless otherwise specified, these terms constitute the common terminology used in this Part and the other Parts of the XBRL GL Next Taxonomy Framework. Compatibility mappings may identify legacy or implementation-specific terms and labels that correspond to these terms.

extension submission
a versioned set of proposed FSM definitions, evidence, mappings, ownership data, tests, and requested reuse classification.

Aligned Pool
the governed aligned extension registry together with the accepted aligned semantic library.

registry entry
the machine-readable lifecycle and provenance record for one governed extension unit.

semantic library artefact
the authoritative FSM rows and related definition records referenced by a registry entry.

assembly extension
the addition of another formal HMD and its message entry points without modifying an existing HMD.

binding evidence
generated Part 2 and Part 3 artefacts and validation results demonstrating that accepted semantics can be realised consistently.

4 Extension classification

4.1 Shared

A definition MUST be classified as Shared when its meaning and intended reuse are independent of a particular country, regime, law, industry, enterprise, product, or bilateral arrangement. Shared does not mean minimal; it means broadly reusable without a context-specific change of meaning.

4.2 Aligned

A definition MUST be classified as Aligned when it specialises, constrains, or adds semantics in order to maintain explicit correspondence with an international, regional, national, legal, regulatory, or published industry source. The registry MUST identify that source and the nature of alignment.

4.3 Distinct

A definition MUST be classified as Distinct when it is specific to an enterprise, enterprise group, product, bilateral relationship, or closed community and is not intended for governed general reuse. Distinct extensions MUST use namespace ownership that does not imply publication in Shared or the Aligned Pool.

4.4 Classification decision

Classification MUST be based on meaning, ownership, reuse scope, and external alignment. Module names, namespace prefixes, HMD roots, and technical importability are not classification criteria.

5 Governed artefacts

5.1 Aligned extension registry

Field	Purpose	Requirement
aligned_entry_id	Stable registry identifier	yes
name; definition; scope_note	Human-readable semantic statement	yes
classification	Shared, Aligned, or Distinct decision	yes
status	proposed, under-review, accepted, deprecated, withdrawn, or rejected	yes
owner; maintainer	Accountable organisation and current maintainer	yes
source; source_id; source_version	External authority or dictionary traceability	for Aligned
relationships	equivalent-to, specialises, aligned-with, replaces, or conflicts-with	when applicable
fsm_artifact	Authoritative semantic-library reference	yes
effective_from; deprecated_from	Lifecycle dates	when applicable
change_log	Version, date, editor, rationale, compatibility classification	yes
binding_evidence	Conforming Part 2 and Part 3 message entry points	for publication

5.2 Aligned semantic library

The aligned semantic library MUST contain the accepted FSM definitions referenced by registry entries and preserve the Part 1 FSM column contract, stable semantic identifiers, module ownership, definitions, labels, multiplicities, Association types, and source mappings. Each registry entry MUST reference those authoritative FSM rows.

5.3 Publication information

Every publication MUST identify the registry version, applicable semantic-library artefacts, namespace versions, accepted and deprecated entries, and corresponding Part 2 and Part 3 message entry points. The method and format used to publish this information are implementation-specific.

6 Submission and review procedure

6.1 Submission package

- proposed FSM rows and definition records;
- requested classification and reuse scope;
- evidence that an equivalent Shared or accepted Aligned definition does not already exist;
- source correspondence for every Aligned definition;
- namespace owner, maintainer, version, and licence information;

- examples and expected Specialisation/Graph Walk results; and
- binding evidence or a plan to produce it before publication.

6.2 Semantic review

- Resolve the proposed Class, Attribute, and Association meanings independently of their physical serialisation.
- Search Shared and the current Aligned Pool for equivalent or overlapping definitions.
- Require reuse, harmonisation, or explicit specialisation where duplication would otherwise occur.
- Verify module ownership, identifiers, definitions, datatypes, multiplicities, and Association semantics.
- Record the classification decision and rationale.

6.3 Processing evidence

Accepted candidate FSM sources MUST be processed with the applicable Foundational Semantic Model by Specialisation and then by Graph Walk. The responsible model definer MUST review the candidate LHM and approve one LHM reviewed file. Post-Graph Walk uses that approved LHM reviewed file as its authoritative input.

Post-Graph Walk MUST identify every level-1 HMD, exclude multiplicity `0..0` subtrees, regenerate XPath values from reviewed element QNames, validate each HMD, and produce one formal HMD_for_taxonomy for each effective root. The HMD-for-taxonomy column at position 9 is `value_domain`, replacing the LHM-only `association_role`. Validation and taxonomy closure are performed per HMD / per DTS; cross-DTS equality is required only for deliberately shared declarations.

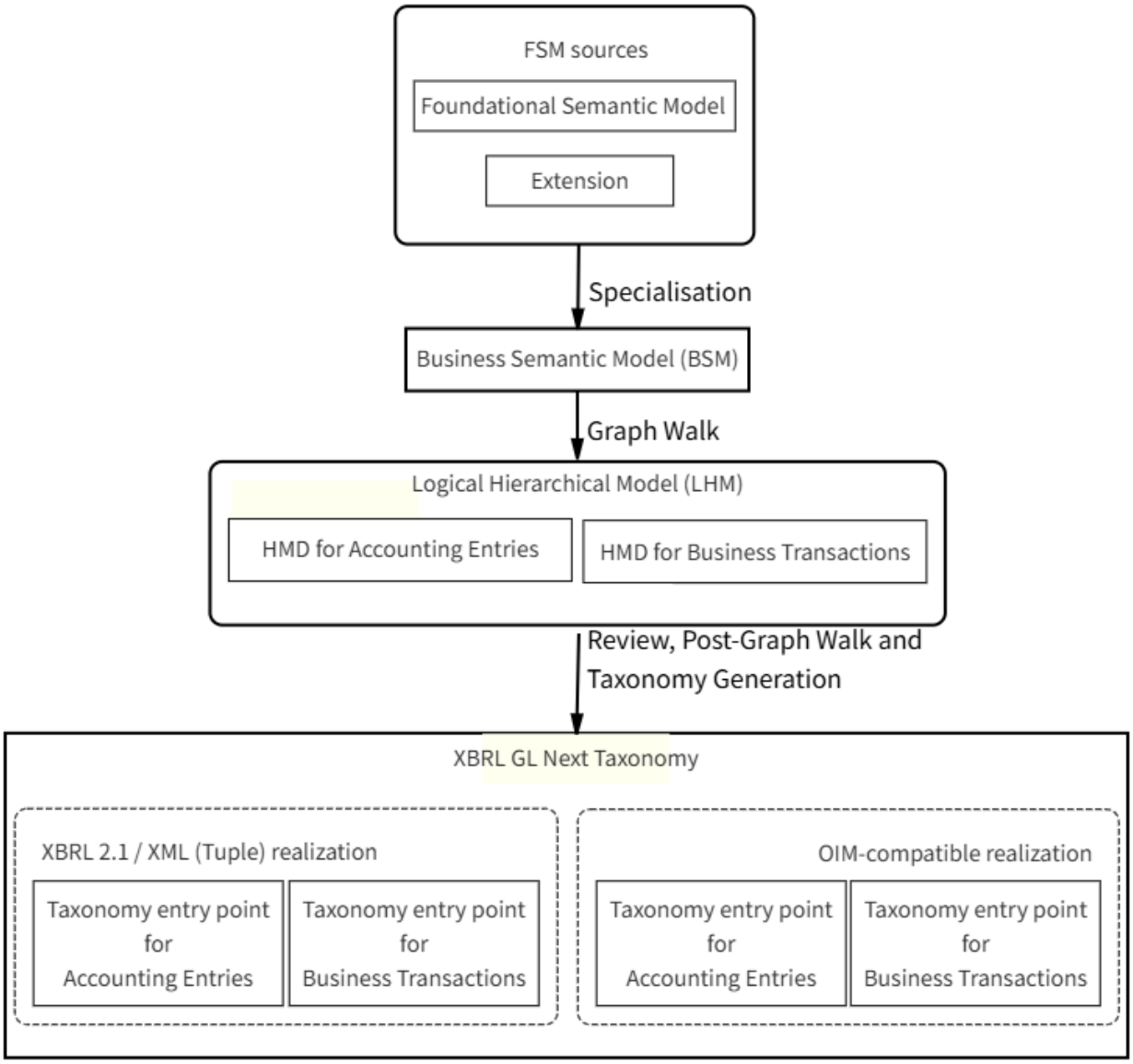


Figure 1 — Layered processing and independent taxonomy realisations

Figure 1 is based on the layered semantic-model Figure used in the Framework. It makes the processing boundary explicit: LHM review produces the authoritative LHM reviewed; Post-Graph Walk produces the formal HMD inputs; and Tuple and OIM taxonomy generation then proceed independently from the same HMDs.

6.4 Binding evidence

Before publication, the accepted extension MUST be demonstrably realisable under Parts 2 and 3. Binding evidence MUST identify the formal HMD inputs, generated message entry points, package version, and conformance results. A difference between Tuple nesting and OIM dimensions is not a semantic difference when Part 3 cross-binding equivalence is satisfied.

Binding evidence generated with the maintained reference implementation SHALL identify `tools/taxonomy/xBRLGL_TaxonomyGenerator.py` as the Generator authority. Its required supporting modules do not constitute a second Generator authority.

7 Lifecycle and compatibility

7.1 Versioning

Every accepted registry entry and semantic-library artefact MUST be versioned. A release MUST distinguish backward-compatible documentation or optional additions from breaking changes to identity, datatype, required minimum occurrence, meaning, or structural content model.

7.2 Deprecation and withdrawal

Deprecation MUST preserve the stable identifier, replacement reference if any, rationale, effective date, and earlier published versions. Implementations MUST be able to reproduce the extension set that applied to an earlier taxonomy version.

7.3 Namespace and release date

Published taxonomy namespaces MUST use the date version required by Parts 2 and 3. The publication information MUST identify the relationship between the registry version and taxonomy namespace version when they differ.

8 Worked assembly extension: businessTransactions (informative)

8.1 Baseline

The baseline formal package contains the HMD rooted at gl-cor:accountingEntries. Its formal HMD root row has module cor, type C, local_name accountingEntries, and xpath /xbrli:xbrl/gl-cor:accountingEntries. Its HMD identifier is cor_accountingEntries.

8.2 Extension objective

The worked extension adds a second, independent HMD rooted at gl-btx:businessTransactions. It is an assembly extension: it does not add businessTransactions beneath accountingEntries and does not change the accountingEntries HMD. Any new FSM definitions required by the businessTransactions hierarchy are submitted and classified separately under Clauses 4 to 6.

8.3 Formal HMD root

Column	Value
module	btx
level	1
type	C
name	Business Transactions
local_name	businessTransactions
xpath	/xbrli:xbrl/gl-btx:businessTransactions
HMD identifier	btx_businessTransactions

8.4 Required generated evidence

Binding	Baseline	Additional HMD
Tuple entry point	tuple/cor_accountingEntries /cor-all-<V>.xsd	tuple/btx_businessTransactions /btx-all-<V>.xsd
OIM entry point	oim/cor_accountingEntries /cor-all-oim-<V>.xsd	oim/btx_businessTransactions /btx-all-oim-<V>.xsd
Tuple module presentation	cor/cor-pre-<V>.xml	btx/btx-pre-<V>.xml
OIM module presentation	cor/cor-oim-pre-<V>.xml	btx/btx-oim-pre-<V>.xml

8.5 Shared QName condition

Where two root-specific HMD DTSs deliberately govern one expanded QName as the same shared declaration, its concept-defining values MUST be identical. Merely encountering the same QName in separate independent DTSs does not by itself impose a cross-DTS comparison. HMD-specific Tuple content models and OIM dimensional networks MAY differ between independent DTSs.

8.6 Acceptance information

The publication information MUST identify both formal HMDs, both pairs of message entry points, the shared-module inventory, the conformance result, and the unchanged accountingEntries binding. If businessTransactions uses accepted Aligned definitions, their registry identifiers and versions MUST also be identified.

8.7 Taxonomy assembly evidence

The binding evidence for btx_businessTransactions MUST show that gl-btx:businessTransactions is declared once in the btx module schema, its label resources locate that declaration by schema URI and element id, and the HMD content schema includes the same-namespace btx taxonomy schema. A child concept owned by another module MUST be referenced by QName through a discoverable cross-namespace import.

The btx_businessTransactions Tuple and OIM entry points MUST discover the applicable module label and base presentation linkbases and the businessTransactions presentation-difference linkbase. New btx concepts receive labels in the btx module label linkbase. Reused concepts retain the labels in their owning module. The HMD presentation difference extends or limits the view with locators and arcs; it does not extend a label linkbase and does not create a new concept declaration.

Where the extension uses a custom presentation or dimensional extended link role, the evidence MUST identify the role URI, its roleType declaration, every roleRef, the usedOn value, and the entry points from which the declaration is discoverable. An override or prohibition of a base presentation relationship MUST use the same extended link role and arcrole as the base relationship.

These checks demonstrate that the extension remains additive at the package level: accountingEntries keeps its own root-specific entry points, businessTransactions receives separate entry points, and shared module concepts and networks are reused rather than redefined.

9 Conformance

9.1 Maintainer conformance

- publish the registry, semantic library, publication information, and version history;
- preserve stable identifiers and prior release records;

- apply the classification and non-duplication review;
- publish reproducible processing and binding evidence; and
- prevent a failed or partial binding generation from being represented as an accepted release.

9.2 Extension-user conformance

- identify the exact registry and semantic-library versions used;
- reuse accepted definitions instead of redefining equivalent semantics;
- process extension FSM sources through the Part 1 pipeline;
- use the Part 2 and Part 3 root-specific message entry points generated for the selected HMDs.

9.3 Prohibited practices

- creating taxonomy-only declarations with no governed semantic source when a semantic extension is required;
- changing accountingEntries merely to add the independent businessTransactions HMD;
- using source_bsm_id as QName identity;
- publishing conflicting shared QName definitions; and
- deleting deprecated registry history or silently replacing published artefacts.

Appendix A (informative) Illustrative registry record for the assembly extension

The illustrative classification value assembly-extension describes the packaging change. Individual semantic definitions used by the new HMD still require Shared, Aligned, or Distinct classification.

```
aligned_entry_id: XBRL-GL-NEXT-HMD-BTX-001
name: Business Transactions HMD assembly
classification: assembly-extension
status: proposed
baseline_hmd: cor_accountingEntries
additional_hmd: btx_businessTransactions
root_qname: gl-btx:businessTransactions
fsm_artifacts: [<authoritative references>]
binding_evidence:
  tuple: tuple/btx_businessTransactions/btx-all-<V>.xsd
  oim: oim/btx_businessTransactions/btx-all-oim-<V>.xsd
compatibility: additive; accountingEntries unchanged
```

Appendix B
References

(Reference list under collaborative review. Official XBRL specifications are available at https://specifications.xbrl.org/.)

RFC2119	The Internet Engineering Task Force (IETF). "Key words for use in RFCs to Indicate Requirement Levels, March 1997" Scott Bradner. (See http://www.ietf.org/rfc/rfc2119.txt)
XPath	World Wide Web Consortium (W3C), XML Path Language (XPath) 2.0 (Second Edition), W3C Recommendation, 14 December 2010. (See https://www.w3.org/TR/xpath20/)
SCHEMA-1	World Wide Web Consortium (W3C). "XML Schema Part 1: Structures" (See http://www.w3.org/TR/xmlschema-1/)
TRP	XBRL International Inc. (XII). "XBRL Taxonomy Recognition Process, 28 October 2004" Peter Calvert, and Joseph MacDonald. (See https://xbrl.org/taxonomyrecognition/taxonomy-recognition-acknowledgement-process-2013-04-09.htm)
XBRL	XBRL International Inc. (XII). "Extensible Business Reporting Language (XBRL) 2.1 Specification with corrected errata dated 2 July 2008" Phillip Engel, Walter Hamscher, Geoff Shuetrim, David vun Kannon, and Hugh Wallis. (See https://specifications.xbrl.org/)
XLINK	World Wide Web Consortium (W3C). "XML Linking Language (XLink) Version 1.0." Steve DeRose, Eve Maler, and David Orchard. (See http://www.w3.org/TR/xlink/)
GXLTFTA	XBRL International Inc. (XII). “XBRL GL Taxonomy Framework Technical Architecture 2015” Gianluca Garbellotto, Eric E. Cohen, Masatomo Goto, Toshimitsu Suzuki (See https://www.xbrl.org/int/gl/2015-03-25/GLTFTA-REC-2015-03-25.html)
XBRL GL Framework	XBRL International Inc. (XII). “XBRL Global Ledger Taxonomy Framework 2015“ Eric E. Cohen, Masatomo Goto, Toshimitsu Suzuki (See https://www.xbrl.org/int/gl/2015-03-25/gl-framework-REC-2015-03-25.html)

- UN/CEFACT Core Component Library, applicable reviewed release.

Appendix C
Intellectual property status

Copyright and license terms are under review. This draft is made available through the public project repository for collaborative review. Publication of this draft does not establish additional redistribution or derivative-use rights.

References to external specifications and other third-party materials remain subject to the terms of their respective rights holders.

This draft does not claim approval, endorsement, publication, or official status by XBRL Japan, XBRL Europe, XBRL International, or any other organization.

Appendix D
Document history

2026-09-15 accepted alignment: This section records the current accepted project specification.

Topic	Accepted specification and boundary
Project module namespace	The namespace of module M is exactly https://www.xbrl.or.jp/taxonomy/xbrl-gl-next/M. The URI contains neither an experimental segment nor a date. Namespace prefixes, versioned physical filenames, and directory paths are separate identifiers and do not alter the namespace URI. External XML, XBRL, XLink, ISO, and historical 2015 XBRL GL namespaces remain unchanged.
Model authority and pipeline	The authoritative authored source is the FSM. Specialisation deterministically produces the BSM. Graph Walk produces the candidate LHM. Human review controls the reviewed LHM. Post-Graph Walk produces the root-specific HMDs. The HMD semantic_path, hierarchy, datatype, multiplicity, and value_domain govern Tuple and OIM generation.
Root-specific DTSs	Accounting Entries and Business Transactions are expanded from different root HMDs. Each root is used and validated through its matching Tuple or OIM entry point. The split layout is mandatory; same-named files are not copied over one another and the two DTSs are not mixed without a separate integration design.
Accepted structural differences	Fourteen tracked differences among same-module and same-name declarations are accepted as ROOT_HMD_SPECIFIC_STRUCTURAL_DIFFERENCE. Exact equality of root-derived type, structure, or multiplicity is not required across AE and BT. Each declaration remains subject to its own accepted root HMD and DTS.
Aligned pool boundary	Shared meaning and governed alignment do not imply identical root-derived physical structures. Shared, Aligned, and Distinct decisions are made in the semantic governance process; Tuple content models and OIM dimensional networks remain root-specific.
Extension assembly	An extension selects compatible root-specific entry points and records provenance. It must not overwrite same-named files or silently assemble AE and BT into one DTS. Single-DTS integration is outside the current acceptance scope.

Date	Details
27 July 2026	Previous collaborative review draft.
8 August 2026	Rewritten around HMD-directed processing; defined governed machine-readable artefacts and evidence; aligned the lifecycle with the 18-column HMD pipeline; added the accountingEntries baseline and businessTransactions assembly-extension example.
8 August 2026	v2: Added binding evidence for module-schema locators, content-schema include/import assembly, shared label reuse, businessTransactions presentation extension, and custom extended link role discovery.
8 August 2026	v3: Added the common terminology statement, clarified the single-input Post-Graph Walk boundary and all-or-nothing HMD publication, and added an updated layered-processing Figure based on the supplied Figure.

Date	Details
10 August 2026	v4: Removed mandatory publication manifests, execution records, hashes, diagnostics, and other implementation-specific output; replaced redundant prohibitions with concise result-oriented requirements; and aligned the businessTransactions example with the generalised formal-HMD and Part 2/3 package model.
28 August 2026	Phase 2C: aligned extension processing with the approved semantic `src/**` paths, sole Generator authority, HMD `value_domain`, one-HMD/one-DTS validation scope, and deliberate-shared-declaration cross-DTS comparison rule.
15 September 2026	Restored the accepted 2026-08-29 successor as the base; aligned project module namespaces with the stable URI; recorded root-HMD-specific AE/BT split conformance and the accepted 14 structural differences; aligned the UADC interface boundary and current semantic items where applicable.

Appendix E
(informative)
Correspondence with XBRL GL Next Requirements

Table E.1 maps the applicable requirements of the XBRL GL Next Requirements Specification to their implementation in this Part. The requirement identifiers and names are reproduced from that Specification. This correspondence is informative and does not create additional requirements or assert implementation conformance.

Table E.1 — Mapping of XBRL GL Next Requirements

Requirement ID	Requirement Name	Implementation in Part 4
GL-REQ-0001	Semantic layer separation	Keeps semantic extension definitions in governed FSM sources and processes them through Specialisation, Graph Walk, reviewed LHM and Post-Graph Walk before syntax bindings are assessed (Clauses 5.2 and 6.3).
GL-REQ-0011	Interoperable extension mechanism for domain attributes	Classifies proposed semantic definitions by Shared, Aligned or Distinct reuse scope, meaning and ownership before registering reusable extensions (Clauses 4 and 6).
GL-REQ-0109	Business transaction module root and anchoring	Illustrates the addition of businessTransactions as an independent root HMD and message-entry-point assembly while leaving accountingEntries unchanged (Clause 8).
GL-REQ-0201	Modular taxonomy architecture	Governs extension assembly and requires binding evidence for separate root-specific Tuple and OIM message entry points and their reusable module relationships (Clauses 6.4 and 8).
GL-REQ-0202	Cross-jurisdiction alignment	Requires an Aligned definition to identify the international, regional, national, regulatory or published industry source with which its semantics correspond (Clauses 4.2 and 5.1).
GL-REQ-0203	Extensibility with business-case discipline	Subjects extension submissions to documented scope, source correspondence, duplication review, semantic review and binding evidence or an evidence plan (Clause 6).
GL-REQ-0204	Localisation ownership, governance, and aligned extension registry	Defines the Aligned extension registry, semantic library, ownership, maintenance, provenance, lifecycle and publication information for governed extensions (Clauses 5 and 7).
GL-REQ-0205	Backward compatibility	Requires versioned registry entries, compatibility classification, preservation of deprecation and replacement history, and reproducibility of previous extension sets (Clause 7).
GL-REQ-0207	Profile declaration of roots and module combinations	Requires published extension information to identify applicable formal HMDs and their Part 2 and Part 3 message entry points; the businessTransactions example demonstrates that assembly (Clauses 5.3 and 8).
GL-REQ-0208	Alignment and extension discipline for btx, sta, and lnk	Distinguishes Shared, Aligned and Distinct semantics on meaning and external alignment rather than module names or importability, including extension governance relevant to domain modules (Clauses 4–6).

GL-REQ-0209	FSM reuse and prohibition of semantic duplication	Requires checking Shared and accepted Aligned definitions for equivalents before adding new FSM definitions, with reuse or explicit specialisation to avoid semantic duplication (Clauses 6.1–6.2).
GL-REQ-0210	Aligned pool of reusable semantic components	Defines an Aligned Pool as a governed registry linked to an accepted FSM semantic library, with reusable identifiers, provenance and versioned lifecycle metadata (Clauses 3 and 5).
GL-REQ-0301	Support for XML and OIM	Requires accepted semantic extensions to be demonstrably realisable through both XBRL 2.1 Tuple and OIM bindings, with their distinct entry points and outcomes documented (Clause 6.4).
GL-REQ-0303	Deterministic transformability	Requires binding evidence that preserves the accepted HMD semantics while permitting distinct Tuple containment and OIM dimensional structures when the cross-binding equivalence conditions hold (Clauses 6.4 and 8.5).
GL-REQ-0403	Interoperability evidence and pilot criteria	Identifies processing and binding evidence, conformance results, versions and publication information needed to substantiate an accepted extension without portraying a partial or failed binding as released (Clauses 6 and 9).